Week 3 Extra: A* Search

Distinction-tier supplement to the greedy best-first lecture

FIT1061: Introduction to Artificial Intelligence

2026-07-27

Table of contents

1 Why this supplement exists 1

2 Where greedy failed 2

3 The A* algorithm 2
3.1 From greedy to A*: what actually changes . . . 3

4 Running A* on the 5-node graph 3

5 Why it broke: the hint overestimated 4

6 Admissibility 4

7 The repair: stop the hint overestimating 4

8 Scaling up to a grid 5

9 When admissibility breaks 6

10 A bigger test: maze-like grid 6

11 Consistency and dominance 7

12 Aiming for a Distinction: what to do 7
12.1 Aiming higher (High Distinction) . . . 8

13 Going further 8

14 Optional viewing 8

15 References 8

1 Why this supplement exists

The Week 3 lecture covers **greedy best-first search** in full. That's enough to complete the Pass task this week.

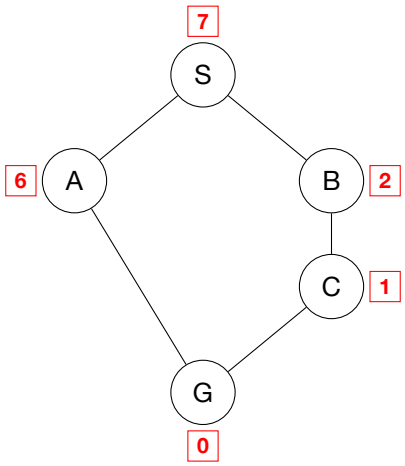
These supplementary notes cover the next step (**A***) which the lecture introduces only as a one-line teaser. If you're aiming for a Distinction, this is required reading. The Distinction task asks you to implement A*, design two admissible heuristics, argue why they never overestimate, and measure how much search a good heuristic saves.

We'll develop A* on a **grid** here (the `GridWorld` from Week 2's BFS example, which you also use this week) because a grid is the easiest place to *see* how g, h, and f interact. Treat the grid as a **practice sandbox**. The graded Distinction task then applies the identical algorithm to a different domain, the **8-puzzle** (a 3×3 sliding-tile puzzle); you'll build it there. Learn the machinery on the grid, then re-home it to the puzzle.

If you've already met A* through other reading, you can skim. The point of these notes is to pin the ideas down precisely and show where the algorithm can go wrong.

2 Where greedy failed

The lecture's worked example is a 5-node graph. We reproduce it here in full so these notes stand on their own.



Edges (each costs 1): S–A, S–B, B–C, C–G, A–G. So there are two ways to the goal: the short path **S–A–G** (2 edges, true cost 2) and the long path **S–B–C–G** (3 edges, true cost 3).

Heuristic values (the "hint" at each node, its guess of the remaining cost to G):

Node	S	A	B	C	G
h	7	6	2	1	0

Greedy best-first always steps to the neighbour with the smallest h. From S it compares A ($h = 6$) against B ($h = 2$), takes the misleadingly-low B, then C ($h = 1$), then G, returning **S–B–C–G** (3 edges). It never tried the high-h neighbour A, which sat one short hop from the goal.

The issue is structural, not bad luck:

> Greedy only considers the *guess* of remaining cost. It never asks how much we've already paid to get here.

A path through a low-h intermediate node can still be long, because the cost-so-far ($g(n)$) accumulates step by step. Greedy ignores $g(n)$ entirely.

A* is built to fix that, but only when the hint never overestimates the true remaining cost.

3 The A* algorithm

A* search expands the node n with the lowest value of

$$f(n) = g(n) + h(n)$$

where

- $g(n)$ is the **cost so far**: the path length from `start` to n along the cheapest known route.
- $h(n)$ is the **heuristic** estimate of cost from n to `goal`.
- $f(n)$ is the **estimated total cost** of the cheapest solution path passing through n.

In other words, A* picks the node whose *total* estimated journey looks shortest. Greedy picks the node that looks close to the goal; uniform-cost search picks the node we reached cheaply; A* weighs both at once.

3.1 From greedy to A*: what actually changes

You already built greedy best-first search at Pass level: a priority queue that always pops the frontier node with the smallest $h(n)$. A* is that *same* search with a few deliberate changes. Writing them yourself is the task, so what follows is a structural map, not code to copy.

- **Track the cost so far.** Alongside the frontier, keep a running g-value for every node reached: the cheapest number of steps from `start` to that node so far. `start` is 0; each move adds 1 (for weighted edges, add the edge's cost instead). Greedy never recorded this.
- **Order the frontier by $f = g + h$, not h alone.** A node's priority when you push it is its cost-so-far plus its heuristic estimate. This one change is what makes A* weigh the road already travelled against the road still ahead.
- **Re-open a node when a cheaper route to it appears.** If you reach an already-seen node by a strictly cheaper path, lower its g, repoint its `came_from` link, and push it again at the new (smaller) f.

Everything else carries straight over from greedy: the priority-queue interface, the `came_from` chain you walk backwards to rebuild the path, and the goal test on pop. Stop when you pop the goal; if the frontier empties first, there is no path.

Two wrinkles worth knowing before you build it from scratch:

- **Stale duplicates.** When your priority queue can't *decrease* a key (a binary heap, e.g. Python's `heapq`, cannot), the re-open step pushes a **fresh copy** of the node rather than updating the old entry, so one node can sit in the frontier more than once. Guard against acting on a stale copy: skip any popped node whose recorded cost-so-far is already **smaller** than the popped entry's g: it's an outdated duplicate.
- **Repeated states need a closed set.** On a domain where the same state is reached by many routes (the 8-puzzle, where countless move sequences land on one tile configuration), keep a **closed set** of settled states and refuse to expand any state twice. That shortcut is safe only under a slightly stronger condition on the heuristic, spelled out in the *Consistency and dominance* box below.

4 Running A* on the 5-node graph

Now run the A* rule (always pop the frontier node with the smallest $f = g + h$) on exactly the graph and hints above. Each edge costs 1, so g is just "edges walked so far".

pop	g	h	f	frontier after expanding
(init)	—	—	—	S(7)
S	0	7	7	A(7), B(3)
B	1	2	3	A(7), C(3)
C	2	1	3	A(7), G(3)
G	3	0	3	goal reached

Reading it step by step: we push S with $f = 0+7 = 7$. Popping S gives two neighbours: A at $f = 1+6 = 7$ and B at $f = 1 + 2 = 3$. The smaller f is B, so we pop B next and push C at $f = 2 + 1 = 3$. Then C, pushing G at $f = 3 + 0 = 3$. We pop G and stop.

A* returns **S–B–C–G, cost 3**, the *longer* path. And look at the frontier column: node A sat there unexpanded at $f = 7$ the entire time, never competitive, even though it was the doorway to the cheaper route.

A* got fooled in exactly greedy's spot. Adding g to the priority did **not**, on its own, save us.

5 Why it broke: the hint overestimated

Circle node A. Its hint says $h(A) = 6$, but the true remaining cost from A is **1**: a single edge, A–G. And $h(S) = 7$, while the true cheapest distance from S to the goal is only **2** (via S–A–G). Both hints **overestimate** the real remaining cost.

A hint that overestimates makes a genuinely cheap node (A, S) *look* expensive, so its f balloons above the decoy path's f and A* never pops it in time. A heuristic that can overestimate this way is called **inadmissible**, and inadmissibility is what broke the search.

6 Admissibility

The shortest-path guarantee holds only if the heuristic is **admissible**, the property our 5-node hints violated.

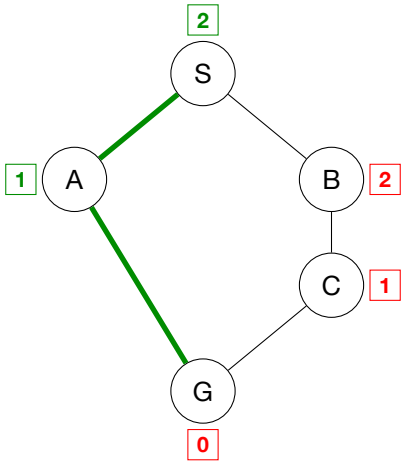
> Definition: admissibility
>
> A heuristic h is **admissible** if, for every node n,
>
> $$h(n) \;\leq\; \text{true cost of the cheapest path from } n \text{ to the goal.}$$
>
> That is, h never *overestimates*.

Why admissibility guarantees the shortest path. Suppose A* is about to return a solution of cost C: it is holding the goal at the front of the frontier at $f = C$. Could a *cheaper* path, of some cost $C' < C$, still be hiding somewhere? If such a path existed, walk along it from the start: the start has already been expanded, the goal has not, so somewhere on that path there is a **first node still sitting unexpanded in the frontier**. Call it n. Its priority is $f(n) = g(n) + h(n)$, where $g(n)$ is its *real* cost so far along that cheap path. Because an admissible h never overestimates, $f(n) = g(n) + h(n) \leq$ (real cost so far) + (real remaining cost) $= C' < C$. So n is sitting in the frontier at $f(n) < C$, and A* pops the smallest f first, so it would have popped n **before** the goal. The cheap path cannot hide: any node that could lead to it is always more attractive than the goal until it has been explored. This argument assumes A* may **re-open** a node when a cheaper route to it turns up, exactly what the pseudocode's `new_cost < cost_so_far[n]` guard allows. The moment you switch to a **closed set** that never re-expands a settled state (as the 8-puzzle implementation does), admissibility alone stops being sufficient, and you need the stronger property of *consistency*, spelled out in the *Consistency and dominance* box below.

This is exactly what failed on the 5-node graph above. There h was **not** admissible, so node A, the doorway to the true cost-2 path, could sit stranded in the frontier at $f = 7$ while A* popped the goal at $f = 3$ and returned the longer route. Remove the overestimate and the argument above applies; leave it in and the guarantee collapses. (The detailed proof is in Russell & Norvig §3.5.)

7 The repair: stop the hint overestimating

The graph never changed and the algorithm never changed. Only two numbers were too high. Fix just those two ($h(A)$ down from 6 to its true remaining cost **1**, $h(S)$ from 7 to its true remaining cost **2**), and leave $h(B) = 2$, $h(C) = 1$, $h(G) = 0$ exactly as they were. The two repaired values are shown in green below, with the shortest path they unlock:



Rerun the identical A*:

pop	g	h	f	frontier after expanding
(init)	—	—	—	S(2)
S	0	2	2	A(2), B(3)
A	1	1	2	G(2), B(3)
G	2	0	2	goal reached

This time popping S pushes A at $f = 1 + 1 = 2$ and B at $f = 1 + 2 = 3$. Now A is the smaller f, so A* pops it, reaches G at $f = 2$, and returns **S–A–G, cost 2**, the true shortest path.

> A* fixes greedy, but only when the hint never overestimates. The $f = g + h$ machinery is necessary but not sufficient: admissibility is what makes it work.

8 Scaling up to a grid

The same behaviour shows up on the grids you already know, where it is easier to *watch*. Take the 10×10 grid you used in Week 2's BFS example (with its walls and obstacles, which force greedy to detour, so its path comes out longer than the shortest). Pick `start = (0,0)`, `goal = (9,9)`.

Algorithm	Path length	Nodes expanded
BFS	19 (shortest)	~ 80
Greedy best-first	24 (longer)	~ 25
A* (Manhattan)	**19 (shortest)**	~ 35

A* finds the **same shortest path as BFS** but expands roughly **half** the nodes. The heuristic does real work: it steers the search toward the goal instead of letting BFS radiate outwards uniformly. Unlike greedy, A* does not pay for this speed-up with sub-optimal paths: because the grid heuristics below are admissible, it is fast *and* optimal.

> **The node-expansion figures here are illustrative.** The exact counts in this section and the maze test below depend on the specific obstacle layout of the grid you run on: when you run the reference implementation yourself you will get precise numbers, and they shift as the walls move. Read these figures for the *pattern* (a tighter admissible heuristic expands fewer nodes), not as exact targets to reproduce.

For our 4-direction grid:

- **Manhattan distance** is admissible: any actual path takes at least $|\Delta\text{row}| + |\Delta\text{col}|$ steps, since each step changes either the row or the column by one.
- **Euclidean distance** is admissible too, and on a 4-direction grid it is in fact *looser* than Manhattan: it under-estimates more. (Looser admissible heuristics expand more nodes.)
- A constant zero, $h(n) = 0$, is trivially admissible. Plug it in and A* orders purely by cost-so-far, which is uniform-cost search (Dijkstra's algorithm), and on these unit-cost grids behaves exactly like BFS.

The general rule: **a tighter admissible heuristic produces a more efficient A*.** Compare on the 10×10 grid:

Heuristic	Path length	Nodes expanded
Manhattan	19	35
Euclidean	19	48
Zero (= Dijkstra/BFS)	19	80

The shortest path and the code stay the same; only the heuristic changes.

9 When admissibility breaks

What if you cheat and use a heuristic that *over*-estimates? The shortest-path guarantee is gone. We saw inadmissibility fool A* into the wrong branch on the 5-node graph; here is the same failure on a grid, where it costs path *length* rather than the wrong branch.

Try $h_{\text{bad}}(n) = 3 \times \text{manhattan}(n, \text{goal})$. On the same 10×10 grid, A* with h_{bad} may return a path that is **longer than the optimal**. The intuition: an over-eager heuristic prematurely prunes paths whose true cost would have proved competitive. It looks at the inflated f-value, decides those branches are too expensive, and never expands them.

> ⚠ The trade-off you trade away
>
> Inadmissible heuristics can be *faster* than admissible ones: they cut more of the search tree. The price is that A* is no longer guaranteed to return the shortest path. Whether that trade is acceptable depends on the application. In a chess engine it is routine. In a route-planning system that promises shortest distance, it is malpractice.

This is what people mean by **weighted A*** in the literature: deliberately inflated heuristics that buy speed at the cost of guaranteed optimality, with a bounded approximation ratio.

10 A bigger test: maze-like grid

A 15×15 grid with corridors and dead-ends magnifies the difference. The corridors force the search to commit to long detours before discovering that a wall blocks the way; the more aggressively the heuristic steers, the more time we spend in dead-ends.

Algorithm	Path length	Nodes expanded
BFS	41 (shortest)	~ 180
Greedy best-first	53 (longer)	~ 60
A* (Manhattan)	**41 (shortest)**	~ 80

A* still finds the shortest path, with under half BFS's expansions. Greedy expands the fewest, but its path is longer than the optimum.

11 Consistency and dominance

> **i** Going further: consistency and dominance
>
> **Consistency (the closed-set subtlety).** The pseudocode above lets a node be re-opened if a cheaper route to it appears. On the grid that is cheap. But on a domain where the *same* state is reached by many routes (the 8-puzzle you are about to build), re-opening states is wasteful, so you keep a **closed set** of settled states and never expand one twice. Once you do that, *admissibility alone is no longer enough* to guarantee the shortest path: a settled node might later turn out to have had a cheaper route you already discarded. What you need is the stronger property of **consistency** (also called *monotonicity*), a triangle inequality for hints:
>
> $$h(n) \;\leq\; \text{cost}(n, n') + h(n')$$
>
> for every step from n to a neighbour n'. In words: **stepping to a neighbour never drops the guess by more than the step actually costs.** Under consistency, the f-value never decreases along a path, so the first time A* settles a state it has already found the cheapest route to it, and the closed set is safe. Every consistent heuristic is admissible; the converse is not guaranteed.
> **Dominance.** Between two consistent heuristics h_1 and h_2, if $h_2(n) \geq h_1(n)$ at every node (still without overestimating), we say h_2 **dominates** h_1. A dominant heuristic is tighter, and A* with it **expands no more nodes** than with the weaker one. You already saw this pattern with Manhattan beating Euclidean on the grid. This is exactly why the Distinction task asks for a *loose* heuristic and a *tighter* one: you get to watch dominance shrink the node count.
> Both 8-puzzle heuristics you will design (misplaced-tiles and total Manhattan distance) are **consistent**: a single blank move changes any one tile's contribution by at most 1, which is the move's cost. That is why the task's closed set stays safe.

12 Aiming for a Distinction: what to do

Everything above built A* on a grid so you could *see* the machinery. The graded Distinction task applies that same machinery to a different, richer domain: the **8-puzzle**, a 3×3 sliding-tile puzzle where a move slides a neighbour of the blank into the gap. It's small enough that BFS still gives you an optimality reference, but big enough that a good heuristic changes the node count dramatically.

Working from the Distinction starter notebook:

1. **Implement A* from scratch** on the 8-puzzle, using the provided priority queue keyed by $f(n) = g(n) + h(n)$. Each move costs 1, so the g-update is the same idea you saw above; what changes is the *domain*: a node is a tile configuration, and its neighbours are the configurations reachable by one blank move.
2. **Design two admissible heuristics:** a loose one (the number of tiles not yet in their goal cell) and a tighter one (the sum of each tile's Manhattan distance to its goal cell). Check that both read 0 at the solved state and that the tighter one never falls below the looser one.
3. **Argue admissibility in your own words** for each heuristic: reason about what a single blank move can and cannot do to each count, rather than restating the generic definition. (It's the same *kind* of argument you saw for grid-Manhattan above; now make it for the puzzle.)
4. **Compare empirically** across at least 10 random puzzles: run BFS, greedy best-first, and A* with both heuristics; confirm A* matches BFS on path length and report how many nodes the tighter heuristic saves.
5. **Reflect** on why tighter helps, what tighter costs per node, when to stop tightening, and when greedy might still be the right call.

The full brief and marking rubric are in the Distinction task notebook. AI tools are **discouraged** for the core implementation, the heuristics, and the admissibility argument at Distinction level; the task discussion will probe your understanding.

12.1 Aiming higher (High Distinction)

Running A* is the easy part; **designing the hint** is the hard one. Take that to a domain nobody handed you: pick a problem from your own project, invent a heuristic for it, and make the case that it's admissible. Argue it from first principles where you can, or test it hard against known shortest paths where you can't prove it. Then use **dominance** to earn a tighter guess: show that a tighter admissible heuristic expands fewer nodes than a loose one, on the same problem. That is the same skill, now on a domain you built yourself.

13 Going further

A* is nearly sixty years old. The original 1968 paper by Hart, Nilsson and Raphael (IEEE Transactions on Systems Science and Cybernetics, vol. 4 no. 2) introduced the algorithm in the context of robot path-planning at SRI International: the same lab that produced Shakey the robot. Modern variants you may encounter later in your degree:

- **Iterative-deepening A* (IDA*)** trades memory for time on huge state-spaces (e.g., 15-puzzle, Rubik's cube).
- **Anytime A*** returns a (sub-optimal) path quickly, then keeps refining if given more time.
- **D* / D* Lite** handle dynamic environments where edge costs change as you search (robot mapping, real-time strategy games).

A related but distinct paradigm, not an A* variant, is **Monte Carlo Tree Search (MCTS)**, used by AlphaGo. Instead of an admissible $g + h$ estimate of remaining cost, it judges a position with **random simulations** to the end of the game and favours the moves that won most (its loop is select–expand–simulate–backpropagate, with no g, no admissible h). Same instinct, spend the search where it pays off, but different machinery. It's the technique that finally cracked Go.

The throughline is the lecture's claim: **better hints, better search.** Every A* variant above is a different bet about what counts as a good hint and how confidently we should trust it.

14 Optional viewing

If you would like to see A* animated, two external explainers are worth the time (both optional):

- Sebastian Lague, *A* Pathfinding (E01)* (https://www.youtube.com/watch?v=-L-WgKMFuhE). Builds visual intuition for the frontier and how the heuristic steers the search across a grid.
- Computerphile, *A* Search Algorithm* (Mike Pound) (https://www.youtube.com/watch?v=ySN5Wnu88nE). Walks the $f = g + h$ bookkeeping and the admissibility condition by hand.

15 References

- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
- Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.), §3.5–3.6.
- Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving.* Addison-Wesley.